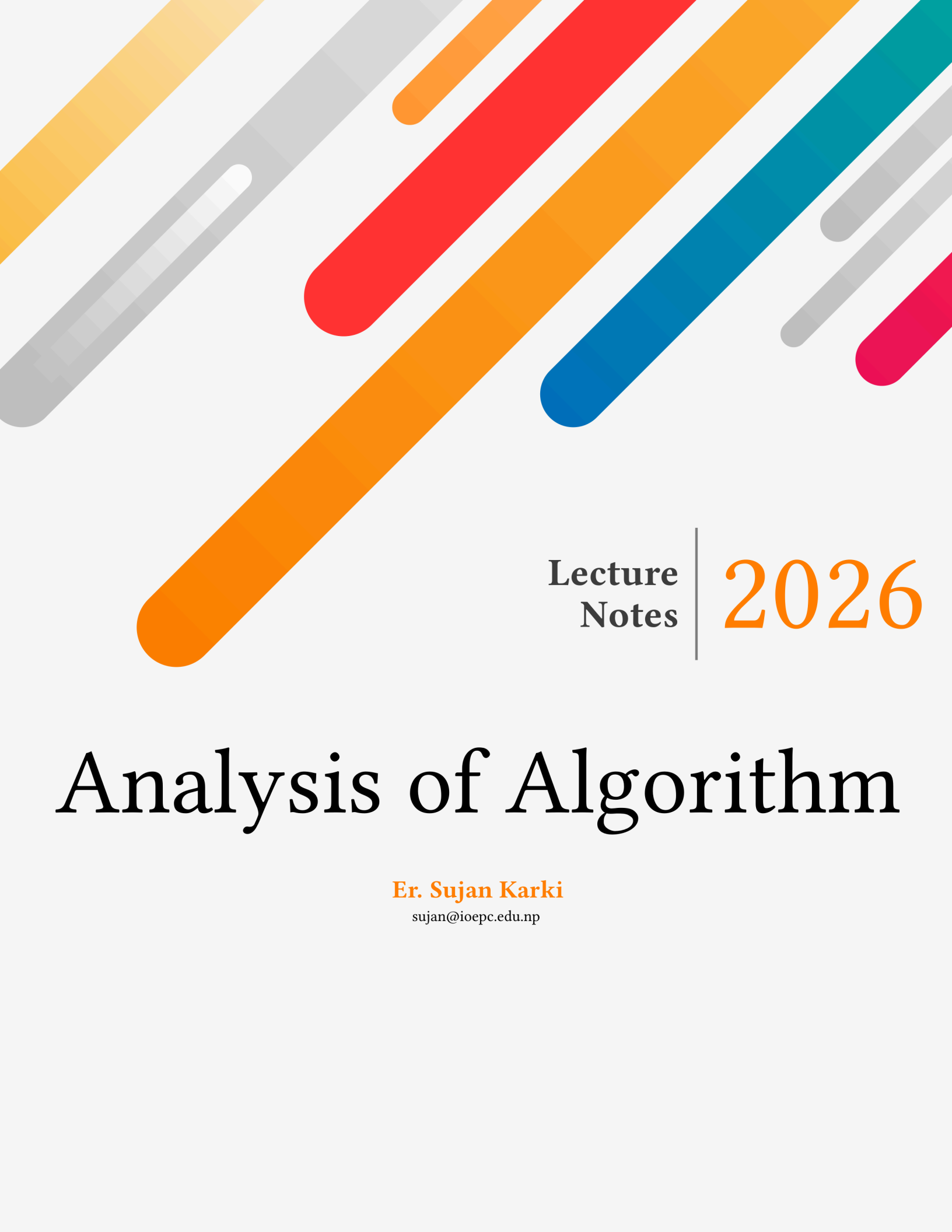




Lecture
Notes

2026

Analysis of Algorithm

Er. Sujan Karki

sujan@ioepc.edu.np

Chapter 5

Dynamic Programming

5.1 Basic Concepts

5.1.1 Introduction

Dynamic Programming (DP) was invented by the U.S. mathematician **Richard Bellman in 1950**. Like greedy algorithms, it is used to solve **optimization problems**. But unlike the greedy approach, dynamic programming **always guarantees the optimal (best) solution**.

Definition 5.1.1

Dynamic Programming is a powerful design technique for solving optimization problems by breaking the problem into smaller **overlapping subproblems**, solving each subproblem only once, storing the result in a table, and reusing it whenever the same subproblem occurs again.

Note:-

The term “programming” here refers to *planning* or systematically constructing a solution. It has *no* direct relation to computer programming or coding.

5.1.2 How Dynamic Programming Works?

Like the divide-and-conquer approach, dynamic programming breaks a problem into smaller subproblems. The key difference:

- In **divide and conquer**, subproblems are *independent*; they are solved recursively and combined.
- In **dynamic programming**, subproblems are *overlapping*; the same subproblems occur multiple times.

Instead of solving overlapping subproblems repeatedly, DP stores the results of solved subproblems in a table. When the same subproblem is encountered again, the answer is retrieved from the table by a simple **lookup procedure**. DP can be implemented in two ways: **top-down with memoization** (recursive) or **bottom-up** (iterative). In the bottom-up approach, it finds the solution of the smallest problem first and constructs the solution of larger problems from already-solved smaller problems.

5.1.3 Dynamic Programming vs Greedy Algorithm

Dynamic Programming	Greedy Approach
Guarantees an optimal solution.	Does not guarantee an optimal solution.
Subproblems are overlapping.	Subproblems do not overlap.
Does more work due to splitting the problem at every possible point.	Does little work due to splitting the problem at specific points only.
Considers future choices while making the current choice.	Considers only the current choice without looking at the future.
There is no specialized set of feasible choices.	Constructs the solution from the set of feasible solutions.
Selects the choice which is <i>globally</i> optimum.	Selects the choice which is <i>locally</i> optimum.
Employs memoization.	No concept of memoization.

Examples of DP: Make-a-change problem, Knapsack problem, Assembly line scheduling, Multistage graph, Matrix chain multiplication.

Examples of Greedy: Binary & fractional knapsack, Job scheduling, Activity selection, Huffman coding, Optimal storage on tapes, Optimal merge pattern.

5.1.4 Recursion vs Dynamic Programming

Recursion	Dynamic Programming
A technique where a function calls itself to solve smaller instances of a problem.	An optimization technique that solves each subproblem once and stores its result.
Follows a top-down approach based directly on the problem's mathematical definition.	Follows either top-down (memoization) or bottom-up (tabulation).
The same subproblem may be solved many times $\Rightarrow$ redundant computations.	Each subproblem is solved once and reused $\Rightarrow$ no redundancy.
Does not handle overlapping subproblems efficiently.	Specifically designed to handle overlapping subproblems efficiently.
Solutions are simple and easy to write.	Solutions are more complex to design and implement.
Time complexity is often high, may be exponential.	Time complexity is usually much lower, often polynomial.
Mainly uses stack memory (recursive calls).	Uses extra memory in the form of tables, arrays, or hash maps.

5.1.5 Elements of the DP Strategy

Dynamic programming is based on **two main properties**:

1. Optimal Substructure

A problem has optimal substructure if the optimal solution of the overall problem can be constructed from the optimal solutions of its subproblems. In other words, if the final solution is optimal, then all solutions to the subproblems used to build it must also be optimal.

Because of this property, DP solves a large problem by:

- Solving smaller subproblems optimally.
- Combining their results to get the optimal solution of the bigger problem.

2. Overlapping Subproblems

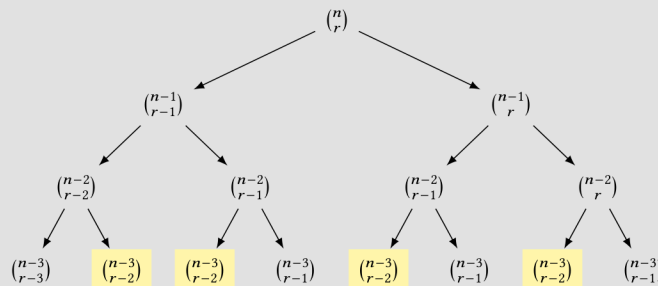
A problem has overlapping subproblems if it can be divided into smaller subproblems and the same subproblems are solved repeatedly. In many recursive solutions, the number of subproblems grows exponentially, but only a *polynomial* number of them are actually distinct.

DP avoids repeated computation by storing each subproblem's result in a table (memoization or tabulation) and reusing the stored result.

Note:-

Example: Binomial Coefficient $C(n, r)$

While computing $C(n, r)$, the subproblem $C(n-3, r-2)$ appears multiple times in the recursion tree. Divide and conquer recomputes $C(n-3, r-2)$ **four times**, leading to redundant work.



DP overcomes this by storing the result of each subproblem. When $C(n-3, r-2)$ is encountered again, the stored value is retrieved directly instead of being recomputed. Thus DP solves each distinct subproblem only **once**.

5.2 All Pair Shortest Path – Floyd–Warshall Algorithm

5.2.1 Problem Statement

The All-Pairs Shortest Path problem (also known as the **Floyd–Warshall algorithm**) is an optimization problem solved using dynamic programming.

Definition 5.2.1

Let $G = \langle V, E \rangle$ be a directed graph, where V is a set of vertices and E is a set of edges with non-negative length. The task is to find the shortest path between **each pair of nodes**.

The Floyd–Warshall recurrence considers whether passing through an intermediate vertex k shortens the path between i and j :

$$D^k[i, j] = \min \left\{ D^{k-1}[i, j], D^{k-1}[i, k] + D^{k-1}[k, j] \right\}$$

where $D^k[i, j]$ is the shortest distance from i to j using only vertices $\{1, 2, \dots, k\}$ as intermediate vertices.

5.2.2 Algorithm

Algorithm 5.2.1

FLOYD_WARSHALL(W, n)

Input: W = adjacency matrix of graph; n = number of vertices

Output: D = matrix of all-pairs shortest distances

1. Step 1: Initialize distance table

For $i \leftarrow 1$ to n :

For $j \leftarrow 1$ to n :

$D[i][j] \leftarrow W[i][j]$

2. Step 2: Update distances using intermediate vertices

For $k \leftarrow 1$ to n :

For $i \leftarrow 1$ to n :

For $j \leftarrow 1$ to n :

If $D[i][j] > D[i][k] + D[k][j]$:

$D[i][j] \leftarrow D[i][k] + D[k][j]$

(intermediate vertex)

(source vertex)

(destination vertex)

3. Return D

5.2.3 Time Complexity

The algorithm has three nested loops:

1. the outer loop over intermediate vertex k runs n times,
2. the middle loop over source i runs n times,
3. and the inner loop over destination j runs n times.

Hence, Total operations = $n \times n \times n = n^3$, i.e. $O(n^3)$.

$$T(n) = O(n^3)$$

5.2.4 Space Complexity

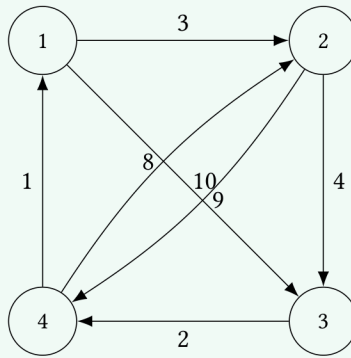
The algorithm primarily operates on a 2D matrix (distance matrix) of size $n \times n$ to store the shortest path distances between every pair of vertices. Hence,

$$S(n) = O(n^2)$$

n is number of vertices

Example 5.1

Consider the following directed graph:



The initial distance matrix D^0 is:

D^0	1	2	3	4
1	0	3	10	∞
2	∞	0	4	9
3	∞	∞	0	2
4	1	8	∞	0

Iteration 1 ($k = 1$): Use vertex 1 as an intermediate vertex. Row 1 and column 1 are copied as-is.

$$D^1[4, 2] = \min\{D^0[4, 2], D^0[4, 1] + D^0[1, 2]\} \\ = \min\{8, 1 + 3\} = 4$$

$$D^1[4, 3] = \min\{D^0[4, 3], D^0[4, 1] + D^0[1, 3]\} \\ = \min\{\infty, 1 + 10\} = 11$$

D^1	1	2	3	4
1	0	3	10	∞
2	∞	0	4	9
3	∞	∞	0	2
4	1	4	11	0

Iteration 2 ($k = 2$): Use vertex 2 as an intermediate vertex. Row 2 and column 2 are copied as-is.

$$D^2[1, 3] = \min\{D^1[1, 3], D^1[1, 2] + D^1[2, 3]\} \\ = \min\{10, 3 + 4\} = 7$$

$$D^2[1, 4] = \min\{D^1[1, 4], D^1[1, 2] + D^1[2, 4]\} \\ = \min\{\infty, 3 + 9\} = 12$$

$$D^2[4, 3] = \min\{D^1[4, 3], D^1[4, 2] + D^1[2, 3]\} \\ = \min\{11, 4 + 4\} = 8$$

D^2	1	2	3	4
1	0	3	7	12
2	∞	0	4	9
3	∞	∞	0	2
4	1	4	8	0

Iteration 3 ($k = 3$): Use vertex 3 as an intermediate vertex. Row 3 and column 3 are copied as-is.

$$D^3[1, 4] = \min\{D^2[1, 4], D^2[1, 3] + D^2[3, 4]\}$$

$$= \min\{12, 7 + 2\} = 9$$

$$D^3[2, 4] = \min\{D^2[2, 4], D^2[2, 3] + D^2[3, 4]\}$$

$$= \min\{9, 4 + 2\} = 6$$

D^3	1	2	3	4
1	0	3	7	9
2	∞	0	4	6
3	∞	∞	0	2
4	1	4	8	0

Iteration 4 ($k = 4$): Use vertex 4 as an intermediate vertex. Row 4 and column 4 are copied as-is.

$$D^4[2, 1] = \min\{D^3[2, 1], D^3[2, 4] + D^3[4, 1]\}$$

$$= \min\{\infty, 6 + 1\} = 7$$

$$D^4[3, 1] = \min\{D^3[3, 1], D^3[3, 4] + D^3[4, 1]\}$$

$$= \min\{\infty, 2 + 1\} = 3$$

$$D^4[3, 2] = \min\{D^3[3, 2], D^3[3, 4] + D^3[4, 2]\}$$

$$= \min\{\infty, 2 + 4\} = 6$$

D^4	1	2	3	4
1	0	3	7	9
2	7	0	4	6
3	3	6	0	2
4	1	4	8	0

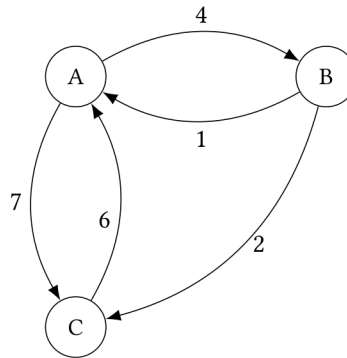
Final Result – Shortest distances between all pairs:

$1 \rightarrow 1 = 0$	$1 \rightarrow 2 = 3$	$1 \rightarrow 3 = 7$	$1 \rightarrow 4 = 9$
$2 \rightarrow 1 = 7$	$2 \rightarrow 2 = 0$	$2 \rightarrow 3 = 4$	$2 \rightarrow 4 = 6$
$3 \rightarrow 1 = 3$	$3 \rightarrow 2 = 6$	$3 \rightarrow 3 = 0$	$3 \rightarrow 4 = 2$
$4 \rightarrow 1 = 1$	$4 \rightarrow 2 = 4$	$4 \rightarrow 3 = 8$	$4 \rightarrow 4 = 0$

Question 5.1

Assignment: All-Pairs Shortest Paths

Consider a directed graph with vertices A , B , and C and the following edge weights:



Task: Compute the shortest path distances between *every pair* of vertices. Show each distance matrix after each iteration.

5.3 Travelling Salesperson Problem (TSP)

5.3.1 Problem Statement

Definition 5.3.1

Given a set of n cities and the distance between each pair of cities, find the **minimum length path** such that it covers each city **exactly once** and terminates the tour at the starting city.

A salesman must visit n cities, making a tour that visits each city exactly once and finishes at the starting city. There is a non-negative cost $d(i, j)$ to travel from city i to city j . The goal is to find a tour of minimum cost. This problem is solved by finding the **Hamiltonian cycle** of minimum weight.

NP-Completeness: There exist $n!$ possible paths, so a search of the optimal path becomes very slow when n is large. TSP is an **NP-complete** problem.

5.3.2 Recurrence Relation

Let V be the set of cities, i the starting vertex, and $C(i, V)$ the minimum travel cost. Then:

$$C(i, V) = \min_{j \in V} \left\{ d[i, j] + C(j, V - \{j\}) \right\}, \quad j \in V, i \notin V$$

5.3.3 Understanding Time Complexity of DP-TSP

With memoization, every distinct state (i, V) is computed **exactly once**. Therefore:

$$\text{Total Work} = (\text{Number of States}) \times (\text{Work per State})$$

Counting the Number of States (i, V)

A state has two parts. Count them independently and multiply.

Choices for current city i

The current city i can be any of the n cities.

$$\text{Choices for } i = n$$

Choices for the remaining set V

V is a *subset* of the n cities. Each city is either **in** S or **not in** S — that is 2 choices per city, made independently for n cities:

$$\text{Choices for } S = \underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ times}} = 2^n$$

Check with $n = 4$: subsets of $\{1, 2, 3, 4\}$ are

$$\begin{aligned} &\emptyset, \{1\}, \{2\}, \{3\}, \{4\}, \{1, 2\}, \{1, 3\}, \{1, 4\}, \{2, 3\}, \{2, 4\}, \{3, 4\}, \\ &\{1, 2, 3\}, \{1, 2, 4\}, \{1, 3, 4\}, \{2, 3, 4\}, \{1, 2, 3, 4\}. \end{aligned}$$

$$\text{Total} = 16 = 2^4 \checkmark$$

Combined Total

$$\text{Total Number of States} = n \times 2^n$$

Work Done in Each State

At each state (i, V) , the algorithm loops over every city $j \in V$ and tries it as the next city:

- The loop runs $|V|$ times.
- In the worst case, $|V|$ can be as large as n .

Therefore:

$$\text{Work per State (worst case)} = O(n)$$

Putting It Together: Time Complexity

Plug into the master formula:

$$\text{Total Time} = \underbrace{(n \times 2^n)}_{\text{number of states}} \times \underbrace{O(n)}_{\text{work per state}}$$

$$\text{Time Complexity} = O(n^2 \cdot 2^n)$$

Where Each Factor Comes From — One-line Summary

$$\underbrace{n}_{\text{current city } i} \times \underbrace{2^n}_{\text{remaining set } S} \times \underbrace{n}_{\text{loop over } j \in S} = n^2 \cdot 2^n$$

5.3.4 Space Complexity

What do we need to store?

Storage	Size
memo[i][S] table (one entry per state)	$n \times 2^n$
Distance matrix $d[i, j]$	n^2
Recursion stack depth	at most n

The memo table dominates (since 2^n grows much faster than n^2):

$$\text{Space Complexity} = O(n \cdot 2^n)$$

Example 5.3.1

Problem: Solve the traveling salesman problem with the associated cost adjacency matrix using dynamic programming.

	1	2	3	4	5
1	—	24	11	10	9
2	8	—	2	5	11
3	26	12	—	8	7
4	11	23	24	—	6
5	5	4	8	11	—

Start the tour from city 1.

Recurrence used:

$$\text{Cost}(i, V) = \min_{j \in V} \left\{ d[i, j] + \text{Cost}(j, V \setminus \{j\}) \right\}, \quad j \in V, i \notin V.$$

Step 1 – Distance from each city to city 1 (no intermediate city):

$$\text{Cost}(2, \Phi) = d[2, 1] = 8$$

$$\text{Cost}(4, \Phi) = d[4, 1] = 11$$

$$\text{Cost}(3, \Phi) = d[3, 1] = 26$$

$$\text{Cost}(5, \Phi) = d[5, 1] = 5$$

Step 2 – Visiting 1 intermediate city:

$$\text{Cost}(2, \{3\}) = d[2, 3] + \text{Cost}(3, \Phi) = 2 + 26 = 28$$

$$\text{Cost}(2, \{5\}) = d[2, 5] + \text{Cost}(5, \Phi) = 11 + 5 = 16$$

$$\text{Cost}(3, \{4\}) = d[3, 4] + \text{Cost}(4, \Phi) = 8 + 11 = 19$$

$$\text{Cost}(4, \{2\}) = d[4, 2] + \text{Cost}(2, \Phi) = 23 + 8 = 31$$

$$\text{Cost}(4, \{5\}) = d[4, 5] + \text{Cost}(5, \Phi) = 6 + 5 = 11$$

$$\text{Cost}(5, \{3\}) = d[5, 3] + \text{Cost}(3, \Phi) = 8 + 26 = 34$$

$$\text{Cost}(2, \{4\}) = d[2, 4] + \text{Cost}(4, \Phi) = 5 + 11 = 16$$

$$\text{Cost}(3, \{2\}) = d[3, 2] + \text{Cost}(2, \Phi) = 12 + 8 = 20$$

$$\text{Cost}(3, \{5\}) = d[3, 5] + \text{Cost}(5, \Phi) = 7 + 5 = 12$$

$$\text{Cost}(4, \{3\}) = d[4, 3] + \text{Cost}(3, \Phi) = 24 + 26 = 50$$

$$\text{Cost}(5, \{2\}) = d[5, 2] + \text{Cost}(2, \Phi) = 4 + 8 = 12$$

$$\text{Cost}(5, \{4\}) = d[5, 4] + \text{Cost}(4, \Phi) = 11 + 11 = 22$$

Step 3 – Visiting 2 intermediate cities (minimum chosen):

$$\begin{aligned} \text{Cost}(2, \{3, 4\}) &= \min\{d[2, 3] + \text{Cost}(3, \{4\}), d[2, 4] + \text{Cost}(4, \{3\})\} \\ &= \min\{2 + 19, 5 + 50\} = \min\{21, 55\} = 21 \end{aligned}$$

$$\begin{aligned} \text{Cost}(2, \{4, 5\}) &= \min\{d[2, 4] + \text{Cost}(4, \{5\}), d[2, 5] + \text{Cost}(5, \{4\})\} \\ &= \min\{5 + 11, 11 + 22\} = \min\{16, 33\} = 16 \end{aligned}$$

$$\begin{aligned} \text{Cost}(2, \{3, 5\}) &= \min\{d[2, 3] + \text{Cost}(3, \{5\}), d[2, 5] + \text{Cost}(5, \{3\})\} \\ &= \min\{2 + 12, 11 + 34\} = \min\{14, 45\} = 14 \end{aligned}$$

$$\begin{aligned} \text{Cost}(3, \{2, 4\}) &= \min\{d[3, 2] + \text{Cost}(2, \{4\}), d[3, 4] + \text{Cost}(4, \{2\})\} \\ &= \min\{12 + 16, 8 + 31\} = \min\{28, 39\} = 28 \end{aligned}$$

$$\begin{aligned} \text{Cost}(3, \{2, 5\}) &= \min\{d[3, 2] + \text{Cost}(2, \{5\}), d[3, 5] + \text{Cost}(5, \{2\})\} \\ &= \min\{12 + 16, 7 + 12\} = \min\{28, 19\} = 19 \end{aligned}$$

$$\begin{aligned} \text{Cost}(3, \{4, 5\}) &= \min\{d[3, 4] + \text{Cost}(4, \{5\}), d[3, 5] + \text{Cost}(5, \{4\})\} \\ &= \min\{8 + 11, 7 + 22\} = \min\{19, 29\} = 19 \end{aligned}$$

$$\begin{aligned} \text{Cost}(4, \{2, 3\}) &= \min\{d[4, 2] + \text{Cost}(2, \{3\}), d[4, 3] + \text{Cost}(3, \{2\})\} \\ &= \min\{23 + 28, 24 + 20\} = \min\{51, 44\} = 44 \end{aligned}$$

$$\begin{aligned} \text{Cost}(4, \{2, 5\}) &= \min\{d[4, 2] + \text{Cost}(2, \{5\}), d[4, 5] + \text{Cost}(5, \{2\})\} \\ &= \min\{23 + 16, 6 + 12\} = \min\{39, 18\} = 18 \end{aligned}$$

$$\begin{aligned} \text{Cost}(4, \{3, 5\}) &= \min\{d[4, 3] + \text{Cost}(3, \{5\}), d[4, 5] + \text{Cost}(5, \{3\})\} \\ &= \min\{24 + 12, 6 + 34\} = \min\{36, 40\} = 36 \end{aligned}$$

$$\begin{aligned} \text{Cost}(5, \{2, 3\}) &= \min\{d[5, 2] + \text{Cost}(2, \{3\}), d[5, 3] + \text{Cost}(3, \{2\})\} \\ &= \min\{4 + 28, 8 + 20\} = \min\{32, 28\} = 28 \end{aligned}$$

$$\begin{aligned} \text{Cost}(5, \{2, 4\}) &= \min\{d[5, 2] + \text{Cost}(2, \{4\}), d[5, 4] + \text{Cost}(4, \{2\})\} \\ &= \min\{4 + 16, 11 + 31\} = \min\{20, 42\} = 20 \end{aligned}$$

$$\begin{aligned} \text{Cost}(5, \{3, 4\}) &= \min\{d[5, 3] + \text{Cost}(3, \{4\}), d[5, 4] + \text{Cost}(4, \{3\})\} \\ &= \min\{8 + 19, 11 + 50\} = \min\{27, 61\} = 27 \end{aligned}$$

Step 4 – Visiting 3 intermediate cities:

$$\begin{aligned}\text{Cost}(2, \{3, 4, 5\}) &= \min\{d[2, 3] + \text{Cost}(3, \{4, 5\}), d[2, 4] + \text{Cost}(4, \{3, 5\}), d[2, 5] + \text{Cost}(5, \{3, 4\})\} \\ &= \min\{2 + 19, 5 + 36, 11 + 27\} = \min\{21, 41, 38\} = 21\end{aligned}$$

$$\begin{aligned}\text{Cost}(3, \{2, 4, 5\}) &= \min\{d[3, 2] + \text{Cost}(2, \{4, 5\}), d[3, 4] + \text{Cost}(4, \{2, 5\}), d[3, 5] + \text{Cost}(5, \{2, 4\})\} \\ &= \min\{12 + 16, 8 + 18, 7 + 20\} = \min\{28, 26, 27\} = 26\end{aligned}$$

$$\begin{aligned}\text{Cost}(4, \{2, 3, 5\}) &= \min\{d[4, 2] + \text{Cost}(2, \{3, 5\}), d[4, 3] + \text{Cost}(3, \{2, 5\}), d[4, 5] + \text{Cost}(5, \{2, 3\})\} \\ &= \min\{23 + 14, 24 + 19, 6 + 28\} = \min\{37, 43, 34\} = 34\end{aligned}$$

$$\begin{aligned}\text{Cost}(5, \{2, 3, 4\}) &= \min\{d[5, 2] + \text{Cost}(2, \{3, 4\}), d[5, 3] + \text{Cost}(3, \{2, 4\}), d[5, 4] + \text{Cost}(4, \{2, 3\})\} \\ &= \min\{4 + 21, 8 + 28, 11 + 44\} = \min\{25, 36, 55\} = 25\end{aligned}$$

Step 5 – Visiting all 4 intermediate cities:

$$\begin{aligned}\text{Cost}(1, \{2, 3, 4, 5\}) &= \min\{d[1, 2] + \text{Cost}(2, \{3, 4, 5\}), d[1, 3] + \text{Cost}(3, \{2, 4, 5\}), \\ &\quad d[1, 4] + \text{Cost}(4, \{2, 3, 5\}), d[1, 5] + \text{Cost}(5, \{2, 3, 4\})\} \\ &= \min\{24 + 21, 11 + 26, 10 + 34, 9 + 25\} \\ &= \min\{45, 37, 44, 34\} = 34\end{aligned}$$

Backtracking – Recovering the optimal path:

The minimum 34 comes from $d[1, 5] + \text{Cost}(5, \{2, 3, 4\}) = 9 + 25$, so from city 1 we go to city 5.

$$1 \longrightarrow 5$$

At $\text{Cost}(5, \{2, 3, 4\}) = 25$, the minimum is $d[5, 2] + \text{Cost}(2, \{3, 4\}) = 4 + 21$, so from 5 we go to city 2.

$$1 \longrightarrow 5 \longrightarrow 2$$

At $\text{Cost}(2, \{3, 4\}) = 21$, the minimum is $d[2, 3] + \text{Cost}(3, \{4\}) = 2 + 19$, so from 2 we go to city 3.

$$1 \longrightarrow 5 \longrightarrow 2 \longrightarrow 3$$

At $\text{Cost}(3, \{4\}) = 19$, only one choice remains: $d[3, 4] + \text{Cost}(4, \Phi) = 8 + 11$, so from 3 we go to city 4.

$$1 \longrightarrow 5 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4$$

Finally, $\text{Cost}(4, \Phi) = d[4, 1] = 11$ returns to the starting city 1.

$$\boxed{1 \longrightarrow 5 \longrightarrow 2 \longrightarrow 3 \longrightarrow 4 \longrightarrow 1}$$

Verification by direct summation:

$$d[1, 5] + d[5, 2] + d[2, 3] + d[3, 4] + d[4, 1] = 9 + 4 + 2 + 8 + 11 = 34.$$

Hence the minimum-cost Hamiltonian tour starting and ending at city 1 has cost 34, achieved by the path $1 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$.

5.4 String Editing

5.4.1 Problem Statement

Definition 5.4.1

The String Edit algorithm finds the **minimum number of operations** required to convert one string into another.

Allowed operations:

1. Insertion
2. Deletion
3. Replacement (Substitution)

Let $X = x_1x_2 \dots x_m$ (length m) and $Y = y_1y_2 \dots y_n$ (length n). Define $dp[i][j]$ = minimum edits to convert $X[1..m]$ into $Y[1..n]$.

5.4.2 Algorithm

Algorithm 5.4.1

STRING_EDIT_DISTANCE(X, Y, m, n)

Input: Strings X (length m), Y (length n)

Output: Minimum edit distance (Levenshtein distance)

1. **Initialize DP table:**

2. For $i \leftarrow 0$ to m :
 $dp[i][0] \leftarrow i$

3. For $j \leftarrow 0$ to n :
 $dp[0][j] \leftarrow j$

4. **Fill DP table:**

For $i \leftarrow 1$ to m then

For $j \leftarrow 1$ to n :

If $X[i] = Y[j]$:

$dp[i][j] \leftarrow dp[i-1][j-1]$

Else:

$dp[i][j] \leftarrow 1 + \min(\underbrace{dp[i-1][j]}_{\text{deletion}}, \underbrace{dp[i][j-1]}_{\text{insertion}}, \underbrace{dp[i-1][j-1]}_{\text{replacement}})$

5. **Return** $dp[m][n]$

5.4.3 Time Complexity

The **Time Complexity** is $O(m \times n)$, where m and n are the lengths of the two strings.

We define a 2-D table $dp[0..m][0..n]$, where each cell $dp[i][j]$ stores the answer to a subproblem involving the first i characters of X and the first j characters of Y .

Number of cells. The table has dimensions $(m+1) \times (n+1)$, giving

$$(m+1)(n+1) = O(m \cdot n) \text{ cells.}$$

Work done per cell. Each cell is computed using a recurrence that examines a *constant* number of neighbouring

cells (typically three: top, left, and top-left diagonal):

$$dp[i][j] = \begin{cases} dp[i-1][j-1], & \text{if } x_i = y_j, \\ 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]), & \text{otherwise.} \end{cases}$$

This is $O(1)$ work per cell — a constant number of additions, comparisons, and a min operation.

Total time.

$$\text{Total time} = \underbrace{(m+1)(n+1)}_{\text{cells}} \times \underbrace{O(1)}_{\text{work per cell}} = O(m \cdot n).$$

$$T(n) = O(m \cdot n)$$

5.4.4 Space Complexity

The **Space Complexity** is $O(m \times n)$ for the DP table.

To compute $dp[i][j]$ correctly, the straightforward implementation stores *every* cell of the table. So,

$$\text{Memory} = (m+1)(n+1) \text{ integer cells} = O(m \cdot n).$$

$$S(n) = O(m \cdot n)$$

Example 5.4.1

Problem: Compute the edit distance between the strings $X = \text{"kitten"}$ and $Y = \text{"sitting"}$ using the dynamic programming approach. Show the DP table after filling it out.

Given, $X = \text{"cat"}$ and $Y = \text{"car"}$

Let, $dp[i][j]$ = minimum number of edits required to convert the first i characters of X into the first j characters of Y .

We know, $dp[i][0] = i$ and $dp[0][j] = j$.

Initial table (boundary row and column filled):

	∅	c	a	r
∅	0	1	2	3
c	1			
a	2			
t	3			

Completed table:

	∅	c	a	r
∅	0	1	2	3
c	1	0	1	2
a	2	1	0	1
t	3	2	1	1

$$\text{Edit Distance} = dp[3][3] = 1 \text{ (replace 't' with 'r').}$$

Example 5.4.2

Problem: Compute the edit distance between the strings $X = \text{"kitten"}$ and $Y = \text{"sitting"}$ using the dynamic programming approach. Show the DP table after filling it out.

Given, $X = \text{"kitten"}$ and $Y = \text{"sitting"}$

Let, $dp[i][j]$ = minimum number of edits required to convert the first i characters of X into the first j characters of Y .

We know, $dp[i][0] = i$ and $dp[0][j] = j$.

Recurrence:

$$dp[i][j] = \begin{cases} dp[i-1][j-1], & \text{if } X[i] = Y[j] \\ 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]), & \text{otherwise} \end{cases}$$

Initial table (boundary row and column filled):

	∅	s	i	t	t	i	n	g
∅	0	1	2	3	4	5	6	7
k	1							
i	2							
t	3							
t	4							
e	5							
n	6							

Completed table:

	∅	s	i	t	t	i	n	g
∅	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

Sequence of edits:

1. Replace 'k' with 's': kitten → sitten
2. Replace 'e' with 'i': sitten → sittin
3. Insert 'g' at the end: sittin → sitting

Edit Distance = $dp[6][7] = 3$

5.5 0/1 Knapsack Problem

Definition 5.5.1

Given a set of items, each having a weight and a value (profit), find the set of items such that the total weight is $\leq$ the knapsack capacity and the total value earned is **maximized**. In 0/1 knapsack, breaking of items is *not* allowed — either take an entire item or reject it (also called **binary knapsack**).

5.5.1 Recurrence Relation

Let $X = \langle x_1, x_2, \dots, x_n \rangle$ be n items, with weights $W = \langle w_1, \dots, w_n \rangle$ and values $V = \langle v_1, \dots, v_n \rangle$, and knapsack capacity M . Let $V[i, j]$ represent the solution for problem size j using the first i items.

$$V[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ V[i-1, j] & \text{if } j < w_i \\ \max \{V[i-1, j], v_i + V[i-1, j - w_i]\} & \text{if } j \geq w_i \end{cases}$$

5.5.2 Algorithm

Algorithm 5.5.1: DP_BINARY_KNAPSACK

Input: n = Set of items X , set of weights $W[1 \dots n]$, profits $v[1 \dots n]$, and knapsack capacity M

Output: Array $V[0 \dots n][0 \dots M]$, which holds the solution of the problem

1. **for** $i \leftarrow 0$ **to** n **do**
 - (a) $V[i, 0] \leftarrow 0$
2. **for** $j \leftarrow 0$ **to** M **do**
 - (a) $V[0, j] \leftarrow 0$
3. **for** $i \leftarrow 1$ **to** n **do**
 - (a) **for** $j \leftarrow 1$ **to** M **do**
 - i. **if** $W[i] \leq j$ **then**
 $V[i, j] \leftarrow \max \{ V[i-1, j], v[i] + V[i-1, j - W[i]] \}$
 - ii. **else**
 $V[i, j] \leftarrow V[i-1, j]$

5.5.3 Time Complexity

The algorithm uses two nested loops: the outer loop iterates over all n items, and the inner loop iterates over all capacities from 1 to M . Each cell $V[i, j]$ is computed in constant time $O(1)$ using previously computed values $V[i-1, j]$ and $V[i-1, j - W[i]]$. Since there are $(n+1)(M+1)$ cells and each takes constant time to fill. Hence, the total running time is $O(n \times M)$.

$$T(n) = O(n \times M)$$

5.5.4 Space Complexity

The algorithm maintains a 2D table V of dimensions $(n+1) \times (M+1)$ to store the optimal values for all subproblems, giving a total of $(n+1)(M+1)$ cells. Hence, the total space complexity is $O(n \times M)$.

$$S(n) = O(n \times M)$$

The above algorithm will just tell us the maximum value we can earn with dynamic programming. It does not speak anything about which items should be selected. We can find the items that give optimum result using the following algorithm:

Algorithm 5.5.2

```

TRACE_KNAPSACK( $w, v, M$ )
//  $w$  is array of weight of  $n$  items
//  $v$  is array of value of  $n$  items
//  $M$  is the knapsack capacity
 $SW \leftarrow \{ \}$ 
 $SP \leftarrow \{ \}$ 
 $i \leftarrow n$ 
 $j \leftarrow M$ 
while ( $j > 0$ ) do
    if ( $V[i, j] == V[i - 1, j]$ ) then
         $i \leftarrow i - 1$ 
    else
         $V[i, j] \leftarrow V[i, j] - v_i$ 
         $j \leftarrow j - w[i]$ 
         $SW \leftarrow SW + w[i]$ 
         $SP \leftarrow SP + v[i]$ 
    end
end
end

```

Example 5.5.1

Find an optimal solution for following 0/1 Knapsack problem using dynamic programming: Number of objects $n = 4$, Knapsack Capacity $M = 5$, Weights (W_1, W_2, W_3, W_4) = (2, 3, 4, 5) and profits (P_1, P_2, P_3, P_4) = (3, 4, 5, 6).

Solution of the knapsack problem is defined as

Recurrence Relation:

$$V[i, j] = \begin{cases} V[i - 1, j] & \text{if } w_i > j \\ \max(V[i - 1, j], v_i + V[i - 1, j - w_i]) & \text{if } w_i \leq j \end{cases}$$

Item	Weight (w_i)	Value (v_i)
I_1	2	3
I_2	3	4
I_3	4	5
I_4	5	6

Boundary conditions would be $V[0, j] = V[i, 0] = 0$. Initial configuration of the table looks like:

	Item Detail		$j \rightarrow$					
			0	1	2	3	4	5
$i = 0$			0	0	0	0	0	0
$i = 1$	$w_1 = 2$	$v_1 = 3$	0					
$i = 2$	$w_2 = 3$	$v_2 = 4$	0					
$i = 3$	$w_3 = 4$	$v_3 = 5$	0					
$i = 4$	$w_4 = 5$	$v_4 = 6$	0					

Filling the table column by column:**Filling column $j = 1$:**For $V[1, 1]$:

- Here, $i = 1, j = 1, w[i] = w_1 = 2$
- Check: Is $j \geq w[i]$? $1 \geq 2$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[1, 1] = V[1 - 1, 1] = V[0, 1] = 0$

For $V[2, 1]$:

- Here, $i = 2, j = 1, w[i] = w_2 = 3$
- Check: Is $j \geq w[i]$? $1 \geq 3$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[2, 1] = V[2 - 1, 1] = V[1, 1] = 0$

For $V[3, 1]$:

- Here, $i = 3, j = 1, w[i] = w_3 = 4$
- Check: Is $j \geq w[i]$? $1 \geq 4$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[3, 1] = V[3 - 1, 1] = V[2, 1] = 0$

For $V[4, 1]$:

- Here, $i = 4, j = 1, w[i] = w_4 = 5$
- Check: Is $j \geq w[i]$? $1 \geq 5$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[4, 1] = V[4 - 1, 1] = V[3, 1] = 0$

Filling column $j = 2$:For $V[1, 2]$:

- Here, $i = 1, j = 2, w[i] = w_1 = 2, v[i] = v_1 = 3$
- Check: Is $j \geq w[i]$? $2 \geq 2$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$

- $V[1, 2] = \max(V[0, 2], 3 + V[0, 2 - 2]) = \max(0, 3 + V[0, 0]) = \max(0, 3) = 3$

For $V[2, 2]$:

- Here, $i = 2, j = 2, w[i] = w_2 = 3$
- Check: Is $j \geq w[i]$? $2 \geq 3$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[2, 2] = V[2 - 1, 2] = V[1, 2] = 3$

For $V[3, 2]$:

- Here, $i = 3, j = 2, w[i] = w_3 = 4$
- Check: Is $j \geq w[i]$? $2 \geq 4$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[3, 2] = V[3 - 1, 2] = V[2, 2] = 3$

For $V[4, 2]$:

- Here, $i = 4, j = 2, w[i] = w_4 = 5$
- Check: Is $j \geq w[i]$? $2 \geq 5$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[4, 2] = V[4 - 1, 2] = V[3, 2] = 3$

Filling column $j = 3$:

For $V[1, 3]$:

- Here, $i = 1, j = 3, w[i] = w_1 = 2, v[i] = v_1 = 3$
- Check: Is $j \geq w[i]$? $3 \geq 2$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$
- $V[1, 3] = \max(V[0, 3], 3 + V[0, 3 - 2]) = \max(0, 3 + V[0, 1]) = \max(0, 3) = 3$

For $V[2, 3]$:

- Here, $i = 2, j = 3, w[i] = w_2 = 3, v[i] = v_2 = 4$
- Check: Is $j \geq w[i]$? $3 \geq 3$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$
- $V[2, 3] = \max(V[1, 3], 4 + V[1, 3 - 3]) = \max(3, 4 + V[1, 0]) = \max(3, 4) = 4$

For $V[3, 3]$:

- Here, $i = 3, j = 3, w[i] = w_3 = 4$
- Check: Is $j \geq w[i]$? $3 \geq 4$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[3, 3] = V[3 - 1, 3] = V[2, 3] = 4$

For $V[4, 3]$:

- Here, $i = 4, j = 3, w[i] = w_4 = 5$
- Check: Is $j \geq w[i]$? $3 \geq 5$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[4, 3] = V[4 - 1, 3] = V[3, 3] = 4$

Filling column $j = 4$:

For $V[1, 4]$:

- Here, $i = 1, j = 4, w[i] = w_1 = 2, v[i] = v_1 = 3$
- Check: Is $j \geq w[i]$? $4 \geq 2$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$
- $V[1, 4] = \max(V[0, 4], 3 + V[0, 4 - 2]) = \max(0, 3 + V[0, 2]) = \max(0, 3) = 3$

For $V[2, 4]$:

- Here, $i = 2, j = 4, w[i] = w_2 = 3, v[i] = v_2 = 4$
- Check: Is $j \geq w[i]$? $4 \geq 3$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$
- $V[2, 4] = \max(V[1, 4], 4 + V[1, 4 - 3]) = \max(3, 4 + V[1, 1]) = \max(3, 4) = 4$

For $V[3, 4]$:

- Here, $i = 3, j = 4, w[i] = w_3 = 4, v[i] = v_3 = 5$
- Check: Is $j \geq w[i]$? $4 \geq 4$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$
- $V[3, 4] = \max(V[2, 4], 5 + V[2, 4 - 4]) = \max(4, 5 + V[2, 0]) = \max(4, 5) = 5$

For $V[4, 4]$:

- Here, $i = 4, j = 4, w[i] = w_4 = 5$
- Check: Is $j \geq w[i]$? $4 \geq 5$? **No**
- So, use formula: $V[i, j] = V[i - 1, j]$
- $V[4, 4] = V[4 - 1, 4] = V[3, 4] = 5$

Filling column $j = 5$:

For $V[1, 5]$:

- Here, $i = 1, j = 5, w[i] = w_1 = 2, v[i] = v_1 = 3$
- Check: Is $j \geq w[i]$? $5 \geq 2$? **Yes**
- So, use formula: $V[i, j] = \max(V[i - 1, j], v[i] + V[i - 1, j - w[i]])$
- $V[1, 5] = \max(V[0, 5], 3 + V[0, 5 - 2]) = \max(0, 3 + V[0, 3]) = \max(0, 3) = 3$

For $V[2, 5]$:

- Here, $i = 2, j = 5, w[i] = w_2 = 3, v[i] = v_2 = 4$

- Check: Is $j \geq w[i]$? $5 \geq 3$? **Yes**
- So, use formula: $V[i, j] = \max(V[i-1, j], v[i] + V[i-1, j-w[i]])$
- $V[2, 5] = \max(V[1, 5], 4 + V[1, 5-3]) = \max(3, 4 + V[1, 2]) = \max(3, 7) = 7$

For $V[3, 5]$:

- Here, $i = 3, j = 5, w[i] = w_3 = 4, v[i] = v_3 = 5$
- Check: Is $j \geq w[i]$? $5 \geq 4$? **Yes**
- So, use formula: $V[i, j] = \max(V[i-1, j], v[i] + V[i-1, j-w[i]])$
- $V[3, 5] = \max(V[2, 5], 5 + V[2, 5-4]) = \max(7, 5 + V[2, 1]) = \max(7, 5) = 7$

For $V[4, 5]$:

- Here, $i = 4, j = 5, w[i] = w_4 = 5, v[i] = v_4 = 6$
- Check: Is $j \geq w[i]$? $5 \geq 5$? **Yes**
- So, use formula: $V[i, j] = \max(V[i-1, j], v[i] + V[i-1, j-w[i]])$
- $V[4, 5] = \max(V[3, 5], 6 + V[3, 5-5]) = \max(7, 6 + V[3, 0]) = \max(7, 6) = 7$

Completed DP Table:

	Item Detail		$j \rightarrow$					
			0	1	2	3	4	5
$i = 0$			0	0	0	0	0	0
$i = 1$	$w_1 = 2$	$v_1 = 3$	0	0	3	3	3	3
$i = 2$	$w_2 = 3$	$v_2 = 4$	0	0	3	4	4	7
$i = 3$	$w_3 = 4$	$v_3 = 5$	0	0	3	4	5	7
$i = 4$	$w_4 = 5$	$v_4 = 6$	0	0	3	4	5	7

Traceback to find selected items:

Start at $V[4, 5] = 7$.

- $V[4, 5] = 7 = V[3, 5] \Rightarrow$ Item I_4 **not** selected. Move to $V[3, 5]$.
- $V[3, 5] = 7 = V[2, 5] \Rightarrow$ Item I_3 **not** selected. Move to $V[2, 5]$.
- $V[2, 5] = 7 \neq V[1, 5] = 3 \Rightarrow$ Item I_2 **selected**. Move to $V[1, 5-3] = V[1, 2]$.
- $V[1, 2] = 3 \neq V[0, 2] = 0 \Rightarrow$ Item I_1 **selected**. Move to $V[0, 2-2] = V[0, 0]$.
- $V[0, 0] = 0 \Rightarrow$ Stop.

Optimal Solution: Items $\{I_1, I_2\}$ with total weight $2 + 3 = 5$ and maximum value 7.

Question 5.2

Question 1: Solve the following 0/1 Knapsack problem using dynamic programming. Knapsack capacity $W = 4$.

Item	Weight (w_i)	Value (v_i)
I_1	1	2
I_2	2	3
I_3	3	4

Solution: Selected items $\{I_1, I_3\}$ with total weight $1 + 3 = 4$ and maximum value = 6.

5.6 Matrix chain multiplication

5.6.1 Problem Statement

Definition 5.6.1

Given n matrices $A_1, A_2, \dots, A_n$, determine the order of multiplication so that the **total number of scalar multiplications** required to compute the product is **minimized**.

In simple terms: In what order, n matrices $A_1, A_2, A_3, \dots, A_n$ should be multiplied so that it would take a minimum number of computations to derive the result.

Cost of matrix multiplication: Two matrices are compatible only if the number of columns of the first equals the number of rows of the second. For A of size $p \times q$ and B of size $q \times r$, the product requires $p \times q \times r$ scalar multiplications.

Example 5.6.1

Why order matters?

Let $A_1 = 5 \times 4$, $A_2 = 4 \times 6$, $A_3 = 6 \times 2$.

$$(A_1 A_2) A_3 = (5 \times 4 \times 6) + (5 \times 6 \times 2) = 120 + 60 = 180$$

$$A_1 (A_2 A_3) = (5 \times 4 \times 2) + (4 \times 6 \times 2) = 40 + 48 = 88$$

Same result, but vastly different multiplication counts.

5.6.2 Recurrence Relation

With dimension vector $d = \{d_0, d_1, \dots, d_n\}$:

$$m[i, j] = \begin{cases} 0 & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + d_{i-1} \cdot d_k \cdot d_j\} & \text{if } i < j \end{cases}$$

where $m[i, j]$ = least number of multiplications required to multiply the matrix subsequence $A_i \dots A_j$.

5.6.3 Algorithm

Algorithm 5.6.1

MATRIX_CHAIN_ORDER(p)

Input: Sequence $p = \langle p_0, p_1, \dots, p_n \rangle$ of matrix dimensions

Output: Minimum cost table m and split-point table s

1. $n \leftarrow \text{length}(p) - 1$
2. For $i \leftarrow 1$ to n :
 $m[i, i] \leftarrow 0$
3. For $l \leftarrow 2$ to n : (l = chain length)
 - (a) For $i \leftarrow 1$ to $n - l + 1$:
 - i. $j \leftarrow i + l - 1$
 - ii. $m[i, j] \leftarrow \infty$
 - iii. For $k \leftarrow i$ to $j - 1$:
 - $q \leftarrow m[i, k] + m[k + 1, j] + p[i - 1] \cdot p[k] \cdot p[j]$
 - If $q < m[i, j]$ then:
 - $m[i, j] \leftarrow q$ (update minimum cost)
 - $s[i, j] \leftarrow k$ (store optimal split point)
4. **Return** m, s

5.6.4 Time Complexity

From the above iterative algorithm, it is easy to observe that the initialization loop runs in $O(n)$ time. The multiplication is carried out within three nested loops. Hence, the total running time is given by:

$$T(n) = \underbrace{O(n)}_{\text{initialization}} + \underbrace{O(n^3)}_{\text{three nested loops}} = O(n^3)$$

Since the cubic term dominates asymptotically, the overall time complexity is:

$$T(n) = O(n^3)$$

5.6.5 Space Complexity

The algorithm requires storage for:

- Input matrices A and B , each of size $n \times n$: $O(n^2)$
- Output matrix C of size $n \times n$: $O(n^2)$
- A constant number of auxiliary variables (loop counters, temporary sums): $O(1)$

Therefore, the total space complexity is:

$$S(n) = O(n^2) + O(n^2) + O(1) = O(n^2)$$

$$S(n) = O(n^2)$$

Example 5.6.2

Problem: Find the minimum number of multiplications required to multiply: $A_{1 \times 5}$, $B_{5 \times 4}$, $C_{4 \times 3}$, $D_{3 \times 2}$, $E_{2 \times 1}$. Also find the optimal parenthesization.

Given

Dimension array:

$$d = \{d_0, d_1, d_2, d_3, d_4, d_5\} = \{1, 5, 4, 3, 2, 1\}$$

Recurrence relation:

$$m[i, j] = \begin{cases} 0, & \text{if } i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + d_{i-1} \cdot d_k \cdot d_j\}, & \text{if } i < j \end{cases}$$

Length $L = 1$ (Base Case)

$$m[1, 1] = m[2, 2] = m[3, 3] = m[4, 4] = m[5, 5] = 0$$

Length $L = 2$

$$m[1, 2] = m[1, 1] + m[2, 2] + d_0 \cdot d_1 \cdot d_2 = 0 + 0 + 1 \times 5 \times 4 = 20$$

$$m[2, 3] = m[2, 2] + m[3, 3] + d_1 \cdot d_2 \cdot d_3 = 0 + 0 + 5 \times 4 \times 3 = 60$$

$$m[3, 4] = m[3, 3] + m[4, 4] + d_2 \cdot d_3 \cdot d_4 = 0 + 0 + 4 \times 3 \times 2 = 24$$

$$m[4, 5] = m[4, 4] + m[5, 5] + d_3 \cdot d_4 \cdot d_5 = 0 + 0 + 3 \times 2 \times 1 = 6$$

$$s[1, 2] = 1, \quad s[2, 3] = 2, \quad s[3, 4] = 3, \quad s[4, 5] = 4$$

Length $L = 3$

Computing $m[1, 3]$:

$$k = 1 : m[1, 1] + m[2, 3] + d_0 \cdot d_1 \cdot d_3 = 0 + 60 + 1 \times 5 \times 3 = 75$$

$$k = 2 : m[1, 2] + m[3, 3] + d_0 \cdot d_2 \cdot d_3 = 20 + 0 + 1 \times 4 \times 3 = 32$$

$$m[1, 3] = \min\{75, 32\} = 32, \quad s[1, 3] = 2$$

Computing $m[2, 4]$:

$$k = 2 : m[2, 2] + m[3, 4] + d_1 \cdot d_2 \cdot d_4 = 0 + 24 + 5 \times 4 \times 2 = 64$$

$$k = 3 : m[2, 3] + m[4, 4] + d_1 \cdot d_3 \cdot d_4 = 60 + 0 + 5 \times 3 \times 2 = 90$$

$$m[2, 4] = \min\{64, 90\} = 64, \quad s[2, 4] = 2$$

Computing $m[3, 5]$:

$$k = 3 : m[3, 3] + m[4, 5] + d_2 \cdot d_3 \cdot d_5 = 0 + 6 + 4 \times 3 \times 1 = 18$$

$$k = 4 : m[3, 4] + m[5, 5] + d_2 \cdot d_4 \cdot d_5 = 24 + 0 + 4 \times 2 \times 1 = 32$$

$$m[3, 5] = \min\{18, 32\} = 18, \quad s[3, 5] = 3$$

Length $L = 4$ **Computing $m[1, 4]$:**

$$k = 1 : m[1, 1] + m[2, 4] + d_0 \cdot d_1 \cdot d_4 = 0 + 64 + 1 \times 5 \times 2 = 74$$

$$k = 2 : m[1, 2] + m[3, 4] + d_0 \cdot d_2 \cdot d_4 = 20 + 24 + 1 \times 4 \times 2 = 52$$

$$k = 3 : m[1, 3] + m[4, 4] + d_0 \cdot d_3 \cdot d_4 = 32 + 0 + 1 \times 3 \times 2 = 38$$

$$m[1, 4] = \min\{74, 52, 38\} = 38, \quad s[1, 4] = 3$$

Computing $m[2, 5]$:

$$k = 2 : m[2, 2] + m[3, 5] + d_1 \cdot d_2 \cdot d_5 = 0 + 18 + 5 \times 4 \times 1 = 38$$

$$k = 3 : m[2, 3] + m[4, 5] + d_1 \cdot d_3 \cdot d_5 = 60 + 6 + 5 \times 3 \times 1 = 81$$

$$k = 4 : m[2, 4] + m[5, 5] + d_1 \cdot d_4 \cdot d_5 = 64 + 0 + 5 \times 2 \times 1 = 74$$

$$m[2, 5] = \min\{38, 81, 74\} = 38, \quad s[2, 5] = 2$$

Length $L = 5$ **Computing $m[1, 5]$:**

$$k = 1 : m[1, 1] + m[2, 5] + d_0 \cdot d_1 \cdot d_5 = 0 + 38 + 1 \times 5 \times 1 = 43$$

$$k = 2 : m[1, 2] + m[3, 5] + d_0 \cdot d_2 \cdot d_5 = 20 + 18 + 1 \times 4 \times 1 = 42$$

$$k = 3 : m[1, 3] + m[4, 5] + d_0 \cdot d_3 \cdot d_5 = 32 + 6 + 1 \times 3 \times 1 = 41$$

$$k = 4 : m[1, 4] + m[5, 5] + d_0 \cdot d_4 \cdot d_5 = 38 + 0 + 1 \times 2 \times 1 = 40$$

$$m[1, 5] = \min\{43, 42, 41, 40\} = 40, \quad s[1, 5] = 4$$

Cost Table $m[i, j]$

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$
$i = 1$	0	20	32	38	40
$i = 2$		0	60	64	38
$i = 3$			0	24	18
$i = 4$				0	6
$i = 5$					0

Split Table $s[i, j]$

	$j = 1$	$j = 2$	$j = 3$	$j = 4$	$j = 5$
$i = 1$	–	1	2	3	4
$i = 2$		–	2	2	2
$i = 3$			–	3	3
$i = 4$				–	4
$i = 5$					–

Optimal ParenthesizationUsing the s table, we recursively split.

- $s[1, 5] = 4 \Rightarrow (A_1 A_2 A_3 A_4)(A_5) = (ABCD)(E)$
- $s[1, 4] = 3 \Rightarrow (A_1 A_2 A_3)(A_4) = (ABC)(D)$

- $s[1, 3] = 2 \Rightarrow (A_1 A_2)(A_3) = (AB)(C)$
- $s[1, 2] = 1 \Rightarrow (A_1)(A_2) = (A)(B)$

Final optimal parenthesization:

$$\left(\left((AB) C \right) D \right) E$$

Verification of cost:

Step	Operation	Resulting Dimensions	Cost
1	AB	1×4	$1 \times 5 \times 4 = 20$
2	$(AB)C$	1×3	$1 \times 4 \times 3 = 12$
3	$((AB)C)D$	1×2	$1 \times 3 \times 2 = 6$
4	$((((AB)C)D)E$	1×1	$1 \times 2 \times 1 = 2$
Total			40

Minimum number of scalar multiplications:

$$m[1, 5] = 40$$

5.7 Flow Shop Scheduling

Scheduling problems arise in manufacturing, computing, and operations research whenever a set of jobs must share a limited collection of machines. The *flow shop* is one of the most studied scheduling environments because of its clean structure and practical relevance (e.g., assembly lines, multi-stage chemical processes, printed circuit board manufacturing).

Definition 5.7.1

A **flow shop** is a scheduling environment with m machines arranged in series and n jobs, where:

- Each job consists of exactly m operations.
- The i -th operation of every job must be processed on machine M_i .
- Operations of a job must follow the order $M_1 \rightarrow M_2 \rightarrow \dots \rightarrow M_m$.
- All jobs are available at time 0, and storage between machines is unlimited.
- Processing times are known and deterministic.

The objective is to find the job arrangement with the shortest possible total makespan.

Note:-

- With **two machines**, the problem is solved in $O(n \log n)$ time using **Johnson's algorithm**.
- For **more than 2 machines**, the problem is **NP-hard**.
- Goal: minimize the sum of completion times (makespan).

5.7.1 Johnson's Algorithm (Two Machines)

S. M. Johnson (1954) gave an elegant rule that produces an *optimal* schedule for the 2-machine flow shop.

Definition 5.7.2

Let there be n jobs, each with two processing times:

- p_{i1} = processing time of job i on machine M_1
- p_{i2} = processing time of job i on machine M_2

In an optimal schedule, job i *precedes* job j if and only if

$$\min\{p_{i1}, p_{j2}\} \leq \min\{p_{j1}, p_{i2}\}.$$

The rule has a simple operational interpretation:

- If a job has a **small time on** M_1 , schedule it *early* so that M_2 starts working as soon as possible (avoid initial idle time on M_2).
- If a job has a **small time on** M_2 , schedule it *late* so that the final idle time on M_2 is minimized (the last job should not waste M_2 's closing window).
- If two jobs have the same minimum time, their relative order does not affect the optimality of the schedule.

5.7.2 Johnson's Algorithm

Algorithm 5.7.1

Input: $T[]$ = processing time table, n = number of jobs

Output: Q = optimal job sequence

1. Initialize empty schedule Q of size n
2. Set:
 - $left \leftarrow 1$
 - $right \leftarrow n$
3. While there are unscheduled jobs:
 - Find minimum processing time t among all unscheduled jobs
 - If t occurs in column M_1 for job k :
 - Place job k at position $left$ of Q
 - $left \leftarrow left + 1$
 - Else:
 - Place job k at position $right$ of Q
 - $right \leftarrow right - 1$
 - Mark job k as scheduled
4. Return Q

Example 5.7.1

Each of five jobs must pass through machines M_1 then M_2 . Find the optimum sequence.

Job	M_1	M_2
A	4	2
B	5	6
C	9	8
D	7	1
E	3	11

Step-by-step application of Johnson's rule: Step 1. The overall minimum time is 1, belonging to job D on machine M_2 . Since the minimum is in column M_2 , place D in the *last* empty slot.

–	–	–	–	D
---	---	---	---	---

Cross out job D .

Step 2. Among the remaining jobs (A, B, C, E), the minimum time is 2, belonging to job A on M_2 . Place A in the last empty slot.

–	–	–	A	D
---	---	---	---	---

Step 3. Now the minimum among $\{B, C, E\}$ is 3 for job E on M_1 . Place E in the *earliest* empty slot.

E	–	–	A	D
---	---	---	---	---

Step 4. Among $\{B, C\}$, the minimum is 5 for job B on M_1 . Place B in the earliest empty slot.

E	B	_	A	D
---	---	---	---	---

Step 5. Only job C remains. It fills the last empty slot:

E	B	C	A	D
---	---	---	---	---

Optimal sequence: $E \rightarrow B \rightarrow C \rightarrow A \rightarrow D$

Computing the Makespan

Once the sequence is fixed, we compute start and finish times on each machine. Let $f_1(k)$ and $f_2(k)$ denote the finish times of the k -th job in the sequence on M_1 and M_2 respectively. Then:

$$f_1(k) = f_1(k-1) + p_{k,1},$$

$$f_2(k) = \max\{f_1(k), f_2(k-1)\} + p_{k,2},$$

with $f_1(0) = f_2(0) = 0$.

Applying this to our sequence E, B, C, A, D :

Order	Job	p_1	Start M_1	Finish M_1	p_2	Start M_2	Finish M_2
1	E	3	0	3	11	3	14
2	B	5	3	8	6	14	20
3	C	9	8	17	8	20	28
4	A	4	17	21	2	28	30
5	D	7	21	28	1	30	31

$$\text{Makespan}_{C_{\max}} = 31 \text{ time units.}$$

5.7.3 Time Complexity

In each iteration, the algorithm scans both columns of the time matrix to find the smallest entry among the unscheduled jobs.

- Iteration 1 scans $2n$ entries.
- Iteration 2 scans $2(n-1)$ entries.
- Iteration 3 scans $2(n-2)$ entries.
- ...
- Iteration n scans $2(1)$ entry.

Total time:

$$2n + 2(n-1) + 2(n-2) + \cdots + 2(1) = 2 \cdot \frac{n(n+1)}{2} = n(n+1) = O(n^2).$$

$$\text{Time Complexity: } O(n^2)$$

5.7.4 Space Complexity

The data structures used by the algorithm are:

- Schedule queue Q of length $n \rightarrow O(n)$.
- Boolean array of size n to mark scheduled jobs $\rightarrow O(n)$.
- Loop indices, left/right pointers, running minimum $\rightarrow O(1)$.

Total space:

$$O(n) + O(n) + O(1) = O(n).$$

Space Complexity: $O(n)$
